TASK 16: Configure an API Gateway to route client requests to appropriate microservices. Implement basic load balancing so that requests are distributed across multiple instances of a service.

AIM

To configure an API Gateway that routes client requests to appropriate microservices and implements basic load balancing across multiple service instances.

PROBLEM STATEMENT

In a microservices architecture, clients should not directly access individual services. Instead, an API Gateway acts as a single entry point, routing requests and distributing load efficiently among multiple instances of services.

SOFTWARE REQUIREMENTS

- Java 17+
- Spring Boot
- Spring Cloud Gateway
- Maven
- IDE (IntelliJ / Eclipse)
- Postman / Browser

CONCEPT OVERVIEW

- **API Gateway** → Entry point for all client requests
- **Routing** → Direct requests to correct microservice
- **Load Balancing** → Distribute requests across multiple instances
- **Service Instances** → Same service running on different ports

STEP 1: CREATE MICROSERVICE (EXAMPLE: STUDENT SERVICE)

Execution: In **IDE (IntelliJ / Eclipse) → New Spring Boot Project**

- File → New → Spring Boot Project
- Project Name: student-service
- Add Dependency: **Spring Web**

1. Open your IDE (IntelliJ / Eclipse)
2. Create a new **Spring Boot Project**
3. Add dependency:
    - Spring Web

4. Name the project: student-service

This micro service is responsible for handling student-related requests. It will later be called by the API Gateway instead of being accessed directly by the client.

CREATE REST CONTROLLER

Inside project folder:
src/main/java → package → create class StudentController.java

Execution

Create a class:

STUDENTCONTROLLER.JAVA

```
@RestController
@RequestMapping("/students")
public class StudentController {

    @GetMapping
    public String getStudents() {
        return "Response from Student Service - Port: " +
                System.getProperty("server.port");
    }

}
```

Explanation

Defines the API endpoint /students which returns response with port number.

- @RestController → Defines REST API
- /students → Endpoint path
- Displays port number → Helps verify load balancing

STEP 2: RUN MULTIPLE INSTANCES OF MICROSERVICE

Execution

Run the same project in **two instances**:

Configure Port (Instance 1)

src/main/resources/application.properties

IDE → Click **Run ▶**

This starts the first service instance on port 8081.

Instance 1:

server.port=8081

Instance 2:

server.port=8082

Explanation

- **We create** multiple instances **of the same service**
- **This is required for** load balancing
- **Each instance runs independently on a different port**

Run the same application on different ports:

application.properties

server.port=8081

Run another instance:

server.port=8082

HOW TO RUN MULTIPLE INSTANCES (IMPORTANT)

You must run the same application on different ports.

Method 1: Using IntelliJ (Recommended)

☞ Top Menu → Run → Edit Configurations

1. Click + → **Spring Boot**
2. Select main class (StudentServiceApplication)
3. In **Program Arguments**, add:

--server.port=8082

4. Click **Apply** → **Run**

- First instance → Runs on 8081
- Second instance → Runs on 8082
- Both run simultaneously
-

Method 2: Using Terminal (Maven)

☞ Open terminal inside project folder

Execution

Run first instance:

```
mvn spring-boot:run -Dspring-boot.run.arguments=--server.port=8081
```

Run second instance (new terminal):

```
mvn spring-boot:run -Dspring-boot.run.arguments=--server.port=8082
```

Each command starts a separate instance.

Method 3: Using JAR File

☞ Terminal

Execution

```
java -jar student-service.jar --server.port=8081
java -jar student-service.jar --server.port=8082
```

Used in real-world deployment.

Verify Both Instances

☞ Browser / Postman

- http://localhost:8081/students
- http://localhost:8082/students

Expected Output

Response from Student Service - Port: 8081
Response from Student Service - Port: 8082

STEP 3: CREATE API GATEWAY PROJECT

IDE → New Spring Boot Project(project name: api-gateway)

- Project Name: api-gateway
- Add Dependencies:
  - Spring Cloud Gateway
  - Spring Web
  - Spring Cloud LoadBalancer

This project acts as the entry point for all client requests.

ADD DEPENDENCIES (POM.XML)

```
<dependency>
   <groupId>org.springframework.cloud</groupId>
   <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>

<dependency>
   <groupId>org.springframework.cloud</groupId>
   <artifactId>spring-cloud-starter-loadbalancer</artifactId>
</dependency>
```

STEP 4: CONFIGURE ROUTING & LOAD BALANCING(CONFIGURE GATEWAY)

api-gateway → src/main/resources/application.yml

APPLICATION.YML

```
server:
  port: 8080
```

Gateway runs on port 8080.

Same file: application.yml

```
spring:
  cloud:
    gateway:
      routes:
        - id: student-service
          uri: lb://STUDENT-SERVICE
          predicates:
            - Path=/students/**
```

Routes requests from /students to student service.

STEP 5: DEFINE SERVICE INSTANCES MANUALLY(CONFIGURE LOAD BALANCER)

APPLICATION.YML (ADD THIS SECTION)

```
spring:
  cloud:
    loadbalancer:
      instances:
        STUDENT-SERVICE:
          - uri: http://localhost:8081
          - uri: http://localhost:8082
```

Registers multiple instances for load balancing.

STEP 6: MAIN CLASS(Run API Gateway)

IDE → Run ApiGatewayApplication

Now gateway is active and ready to route requests.

Now the system is ready:

- Gateway is running on **8080**
- Services are running on **8081 & 8082**

@SpringBootApplication

```
public class ApiGatewayApplication {
   public static void main(String[] args) {
      SpringApplication.run(ApiGatewayApplication.class, args);
   }
}
```

HOW IT WORKS

- Client sends request → http://localhost:8080/students
- API Gateway intercepts request
- Routes to STUDENT-SERVICE
- Load balancer distributes requests:
  - First → 8081
  - Next → 8082
  - (Round-robin)

STEP 7: TESTING(Test Through API Gateway)

Browser / Postman

You will observe:

- First response → Port 8081
- Next response → Port 8082

This confirms:
✓ Routing is working
✓ Load balancing is working

USING BROWSER/POSTMAN:

http://localhost:8080/students

Refresh multiple times.
You will see alternating responses:
Port 8081
Port 8082
This confirms load balancing.

OUTPUT:

Response from Student Service - Instance running on port: 8081

Response from Student Service - Instance running on port: 8082

Step	Where it Happens
Client Request	Browser / Postman
Routing	API Gateway (Port 8080)
Load Balancing	Gateway LoadBalancer
Processing	Student Service (8081 / 8082)
Response	Back to Client

1. Client sends request to API Gateway
2. Gateway checks route configuration
3. Identifies target service (STUDENT-SERVICE)
4. Load balancer selects one instance
5. Request forwarded to selected instance
6. Response returned to client

ALGORITHM

1. Start multiple instances of microservice
2. Configure API Gateway routes
3. Map service name to instances
4. Use load balancer (round-robin)
5. Receive and forward client request
6. Return response from selected instance

EXPECTED OUTPUT

- Requests are routed through API Gateway
- Load is distributed across service instances
- No direct client access to services

ADVANTAGES

- Centralized request handling
- Improved scalability
- Fault tolerance
- Easy monitoring and security integration

All steps are performed across:

- **IDE** → For coding and running applications

- **application.properties / application.yml** → For configuration
- **Browser/Postman** → For testing

The API Gateway successfully routes and balances requests between multiple service instances.

RESULT

The API Gateway successfully routes client requests and balances load across multiple instances, ensuring scalability and efficient resource utilization in a microservices architecture.